

No. of Functional Features Included in the Solution

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Functional Features Overview:

This document lists all the functional features included in the project solution. Each feature is described with its purpose, implementation status, and contribution to the overall solution.

S.No	Feature Name	Feature Description	Module / Component	Status	Marks Contribution
1	Crop Recommendation	Accepts 9 inputs (N, P, K, temp, humidity, pH, rainfall, location, season) and predicts the best crop from 22 classes using an ensemble of RandomForest, DecisionTree, and SVC models with probability averaging. Returns predicted crop, confidence %, and top 10 candidate crops.	server.py /api/recommend + crop_model.joblib	Done	Core
2	Crop Suitability Assessment	Given a specific crop and 7 environmental values, calculates a weighted suitability score across all 7 parameters using dataset-derived ideal ranges. Returns suitability label (Excellent/Strong/Moderate/Poor), overall score %, per-metric status, and remediation steps.	server.py /api/suitability	Done	Core
3	Fertilizer Recommendation	Analyses crop-specific nutrient requirements vs. current N, P, K values and pH. Recommends fertilizer type (Urea/DAP/MOP/None), provides nutrient balance classification (Low/Balanced/High), and generates a soil pH note.	server.py /api/fertilizer + get_fertilizer_recommendation()	Done	Core
4	Symptom-based Disease Detection	Accepts crop name, temperature, humidity, pH, and a list of observed symptoms. Classifies the most likely disease using a trained Random Forest model and returns predicted disease name, confidence %, and risk level (High/Medium/Low).	server.py /api/disease + disease_model.joblib	Done	Core
5	Image-based Disease Detection	Accepts a raw uploaded leaf image (JPEG/PNG/BMP/GIF/TIFF) and crop name. Classifies the plant disease using a Keras CNN or a pixel-feature Random Forest fallback. Returns predicted disease and confidence %.	server.py /api/image-disease + image_disease_model.joblib / .h5	Done	Core

6	Real-time Weather Auto-fill	Accepts a city name or GPS coordinates (lat/lon). Calls Open-Meteo geocoding + forecast API (no API key required) to return current temperature (°C), humidity (%), and 30-day total rainfall (mm) for automatic form pre-fill.	server.py /api/weather + Open-Meteo API	Done	Core
7	Research Analytics Dashboard	Aggregates crop-environment statistics by crop, region (North/South/Central India), and season (Kharif/Rabi/Zaid). Serves data to insights.html where Chart.js renders bar charts, radar charts, and scatter plots. Users can filter by crop/region/season and export filtered data as CSV.	server.py /api/analytics + insights.html + Chart.js	Done	Additional
8	Responsive Multi-page Web UI	5 HTML pages (index, recommend, disease, insights, contact) with a shared CSS stylesheet, mobile hamburger menu, hero images, feature cards, GPS weather button, mode tabs, and dynamic JS rendering of results without page reload.	web/ folder (HTML, CSS, JS, assets)	Done	Core

Feature Summary:

Metric	Count / Value
Total Features Planned	8
Total Features Implemented	8
Core / Must-Have Features	7
Additional / Nice-to-Have Features	1
Features Tested & Verified	8

Feature Category Breakdown:

S.No	Category	Features in Category	Example Features
1	User Interface (UI)	2	Responsive Web UI (5 pages + shared CSS/JS), Research Analytics Dashboard with Chart.js
2	Backend / Logic	4	Crop Recommendation Engine, Crop Suitability Assessor, Fertilizer Advisor, Disease Classifiers (symptom + image)
3	Database / Storage	1	Flat-file storage: Crop_recommendation.csv + opticrop-data.json + .joblib model files
4	API / Integration	1	Real-time Weather Auto-fill via Open-Meteo API (geocoding + forecast, no key required)
5	Security / Authentication	0	No authentication (open local app). Directory traversal prevention via safe_resolve() in server.py.